

HW1

Ben Schanzer

2026-04-10

Question 1

Generate a dataframe with at least ten named individuals. Give them scores on five assignments. Include some missing assignments. Add one column that gives the mean of all their scores, figuring the mean for those with missing assignments based on the number submitted, not the number assigned.

```
names = c("Fred", "Ashely", "Owen", "Avery", "Andy", "Andrea", "Emily", "Danny", "Ben", "Conor")

scores <- matrix(c(
  91, 90, 69, 82, 89,
  60, 65, NA, 79, 67,
  75, NA, 95, 72, 84,
  98, 73, 61, NA, 76,
  NA, 80, 70, 93, 62,
  87, 64, 78, 96, 71,
  83, 92, 66, 74, NA,
  68, 77, 100, 63, 88,
  81, NA, 86, 70, 94,
  72, 85, 91, NA, 79
), ncol = 5, byrow = TRUE)

df = as.data.frame(scores)
df = cbind(Student = names, df)

# Row averages ignoring NA values
df$row_avg <- rowMeans(as.matrix(df[,-1]), na.rm = TRUE)

colnames(df) = c("Student", "Test 1", "Test 2", "Test 3", "Test 4", "Test 5", "Average Test Score")

df
```

##	Student	Test 1	Test 2	Test 3	Test 4	Test 5	Average Test Score
## 1	Fred	91	90	69	82	89	84.20
## 2	Ashely	60	65	NA	79	67	67.75
## 3	Owen	75	NA	95	72	84	81.50
## 4	Avery	98	73	61	NA	76	77.00
## 5	Andy	NA	80	70	93	62	76.25
## 6	Andrea	87	64	78	96	71	79.20
## 7	Emily	83	92	66	74	NA	78.75
## 8	Danny	68	77	100	63	88	79.20
## 9	Ben	81	NA	86	70	94	82.75
## 10	Conor	72	85	91	NA	79	81.75

Question 2

Using the UKfaculty graph from igraphdata, generate an edgelist and an adjacency matrix. Display the first 10 rows of the edgelist and the upper left 10x10 of the adjacency matrix. Create a separate dataframe listing each node id along with degree, closeness centrality, and betweenness centrality.

```
library(igraphdata)
library(igraph)
```

```
##
## Attaching package: 'igraph'

## The following objects are masked from 'package:stats':
##
##      decompose, spectrum

## The following object is masked from 'package:base':
##
##      union
```

```
data("UKfaculty")
```

```
A = as_adjacency_matrix(UKfaculty, sparse = FALSE)
```

```
## This graph was created by an old(er) igraph version.
## i Call 'igraph::upgrade_graph()' on it to use with the current igraph version.
## For now we convert it on the fly...
```

```
E = as_edgelist(UKfaculty)
```

```
cat("Adjacency Matrix (top-left 10x10):\n")
```

```
## Adjacency Matrix (top-left 10x10):
```

```
print(A[1:10, 1:10])
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,]    0    0    0    1    0    0    0    0    0    0
## [2,]    0    0    0    0    0    0    0    0    0    0
## [3,]    0    0    0    0    0    0    0    0    0    0
## [4,]    1    0    1    0    0    0    0    0    0    0
## [5,]    0    0    0    0    0    1    1    0    1    1
## [6,]    0    0    0    0    0    0    1    0    0    0
## [7,]    0    0    0    0    0    1    0    0    0    1
## [8,]    0    0    0    0    0    0    0    0    0    0
## [9,]    0    0    1    1    0    0    0    0    0    0
## [10,]   0    0    0    0    1    0    1    0    0    0
```

```
cat("\nEdge List (first 10 rows):\n")
```

```
##  
## Edge List (first 10 rows):
```

```
print(E[1:10, ])
```

```
##      [,1] [,2]  
## [1,]  57  52  
## [2,]  76  42  
## [3,]  12  69  
## [4,]  43  34  
## [5,]  28  47  
## [6,]  58  51  
## [7,]   7  29  
## [8,]  40  71  
## [9,]   5  37  
## [10,] 48  55
```

```
unique_nodes = unique(as.vector(E))  
df2 = data.frame(node = unique_nodes)  
df2$degree = degree(UKfaculty)[unique_nodes]  
df2$closeness = closeness(UKfaculty)[unique_nodes]  
df2$betweenness = betweenness(UKfaculty)[unique_nodes]  
  
df2
```

```
##      node degree  closeness betweenness  
## 1      57      24 0.0030211480 142.4654040  
## 2      76      13 0.0020703934  11.5250000  
## 3      12      19 0.0021598272  29.8955128  
## 4      43      33 0.0035971223  77.7957792  
## 5      28      12 0.0028409091  79.3115551  
## 6      58      25 0.0028011204 431.8554695  
## 7       7      35 0.0023584906 245.2856061  
## 8      40      19 0.0032051282 148.1517857  
## 9       5      38 0.0045045045 727.4570360  
## 10     48      14 0.0031152648 325.6341492  
## 11      6      17 0.0031250000 296.0070679  
## 12     21      38 0.0025445293 143.3159452  
## 13     67       6 0.0028011204  88.3321581  
## 14     65      10 0.0020876827   7.3333333  
## 15     52      39 0.0042735043 381.8448163  
## 16     37      54 0.0045045045 1223.0844364  
## 17      4      18 0.0022172949 185.9786797  
## 18     19      24 0.0025706941  16.4607143  
## 19     74      13 0.0015197568   0.0000000  
## 20     62      43 0.0050251256 965.5960470  
## 21     15      29 0.0039370079 104.0853896  
## 22     72      19 0.0030769231 445.4177683  
## 23     46      30 0.0027777778 106.8142857  
## 24      2      36 0.0036764706 711.2849456
```

## 25	22	20	0.0031645570	231.1086081
## 26	71	13	0.0021645022	0.1250000
## 27	77	44	0.0030395137	349.7698732
## 28	23	22	0.0016181230	0.8333333
## 29	20	21	0.0031847134	71.6873016
## 30	34	17	0.0039682540	39.4174603
## 31	51	24	0.0034129693	234.3818043
## 32	38	21	0.0039840637	628.5462482
## 33	36	10	0.0020325203	0.0000000
## 34	17	14	0.0037174721	522.2601732
## 35	44	6	0.0037453184	4.2738095
## 36	18	22	0.0006920415	0.0000000
## 37	29	62	0.0045248869	684.1950577
## 38	10	35	0.0030030030	389.9184094
## 39	68	27	0.0026666667	62.9221001
## 40	49	26	0.0029325513	248.0175075
## 41	30	10	0.0025252525	37.2781136
## 42	53	10	0.0022123894	201.8959957
## 43	73	4	0.0024449878	0.0000000
## 44	75	14	0.0033898305	322.7988456
## 45	78	6	0.0020161290	0.0000000
## 46	27	37	0.0038759690	260.0696484
## 47	66	8	0.0021691974	20.8666667
## 48	26	20	0.0037593985	5.4191198
## 49	31	35	0.0031347962	109.7913781
## 50	69	37	0.0029154519	368.5825411
## 51	24	9	0.0025445293	241.3940629
## 52	13	26	0.0031055901	96.7674714
## 53	64	13	0.0021505376	9.9393939
## 54	61	16	0.0019841270	104.0959957
## 55	42	29	0.0024390244	143.4625763
## 56	50	20	0.0020703934	5.2289683
## 57	14	12	0.0013020833	0.0000000
## 58	16	19	0.0021052632	16.5000000
## 59	70	18	0.0036101083	190.0153666
## 60	81	10	0.0025188917	68.0222222
## 61	33	27	0.0026041667	20.6452742
## 62	32	10	0.0007412898	0.0000000
## 63	35	28	0.0028735632	263.6281746
## 64	79	22	0.0028089888	70.3940476
## 65	3	8	0.0015174507	5.5000000
## 66	47	12	0.0014084507	5.5666667
## 67	39	20	0.0030030030	139.3486416
## 68	55	11	0.0018796992	93.1060606
## 69	1	15	0.0028901734	347.1190476
## 70	9	13	0.0030959752	272.7139250
## 71	54	26	0.0030864198	389.1091187
## 72	45	12	0.0019880716	5.1666667
## 73	80	15	0.0024937656	90.2478175
## 74	8	8	0.0010493179	0.0000000
## 75	63	9	0.0010559662	0.0000000
## 76	56	10	0.0017985612	10.9978175
## 77	25	13	0.0024752475	43.7797619
## 78	59	11	0.0017513135	38.2029942

## 79	41	12	0.0014388489	0.2000000
## 80	60	5	0.0010416667	0.0000000
## 81	11	2	NaN	0.0000000

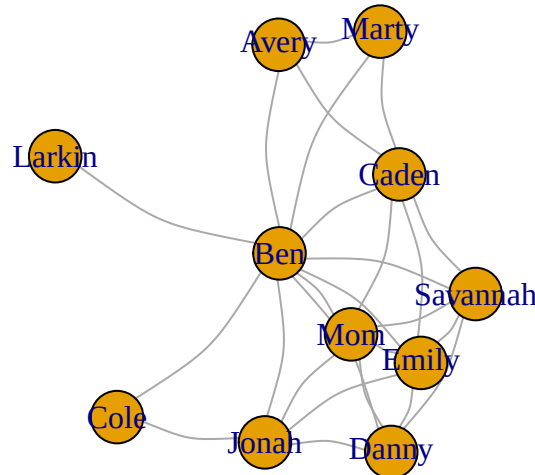
Question 3

Create an edgelist that includes the connections between you and at least 10 friends or family. Create an igraph object from the edgelist. Plot the graph of your connections, including connections among friends and family.

```
edges <- c("Ben", "Jonah",
           "Ben", "Cole",
           "Ben", "Savannah",
           "Ben", "Caden",
           "Ben", "Emily",
           "Ben", "Danny",
           "Ben", "Marty",
           "Ben", "Avery",
           "Ben", "Mom",
           "Jonah", "Cole",
           "Jonah", "Emily",
           "Jonah", "Danny",
           "Jonah", "Mom",
           "Savannah", "Caden",
           "Savannah", "Emily",
           "Savannah", "Danny",
           "Savannah", "Mom",
           "Caden", "Emily",
           "Caden", "Avery",
           "Caden", "Marty",
           "Caden", "Mom",
           "Emily", "Danny",
           "Emily", "Mom",
           "Danny", "Mom",
           "Marty", "Avery",
           "Ben", "Larkin"
)

edge_list <- matrix(edges, ncol = 2, byrow = TRUE)

g = graph_from_edgelist(edge_list, directed = FALSE)
plot(g,
     vertex.size = 25,
     edge.curved = 0.25)
```



Question 4

Research Question:

I am interested in conspiracy theories (CTs) and how they travel and evolve through social networks. Specifically, I want to map networks of conspiracy believers to determine whether there are central nodes — highly connected individuals who act as hubs for spreading new conspiracies through a community. Beyond just the spread of individual theories, I am also curious whether believing in one conspiracy predisposes someone to believe in others. In other words, is someone who believes in Theory A more likely to adopt an unrelated Theory B simply because they are already embedded in a conspiratorial network, or are there other factors — such as political identity, media diet, or personal relationships — that determine which specific theories someone adopts? Understanding this could help researchers identify whether conspiracy belief is driven more by network influence or by individual predisposition.

Data Acquisition:

Data could be acquired by scraping public posts and interactions on platforms like Facebook, Twitter/X, or Reddit, using keyword filtering and natural language processing to identify users expressing conspiratorial beliefs. Connections between users could be established through follower relationships, replies, retweets, or shared group memberships. If the goal is to track the flow of a specific conspiracy theory through a network, a directed graph would be appropriate, since we care about who transmitted the theory to whom. This would also allow us to work backwards from widespread belief to identify the original source or sources. If instead the goal is simply to map the community of conspiracy believers and their relationships, an undirected graph would suffice. Weights could serve multiple purposes: on edges, they could represent the frequency or intensity of interaction between two individuals, while node attributes could track how many distinct theories each person believes. Regarding components, the graph would likely have one large connected component encompassing the main conspiracy community, but there may be smaller isolated components representing

niche or fringe groups that do not interact with the broader network — for example, a small group focused on a hyper-local conspiracy that has no overlap with mainstream CT communities.

Question 5

Find the definition of the diameter of a graph. Find the igraph function for measuring diameter. Using the e-mail network as an undirected graph: simplify the network, determine its diameter, and use bootstrapping to determine whether the diameter has a statistically significant difference from a null model of a random graph with the same degree distribution.

Diameter Definition: The diameter of a connected (un)directed graph is the farthest distance between any two of its vertices.

```
ecomp = as.data.frame(read.table(file = "../Data/email_main_component.txt", sep = "|"))
email_g = graph_from_data_frame(ecomp, directed = FALSE)
email_g = simplify(email_g)

cat("Email network diameter:", diameter(email_g, directed = FALSE), "\n")

## Email network diameter: 16

# Bootstrapping: compare diameter to null model (random graph with same degree distribution)
d1 = degree(email_g)
t1 = numeric(1000)

for (i in 1:1000) {
  randnet = sample_degseq(d1)
  randnet = simplify(randnet)
  t1[i] = diameter(randnet, directed = FALSE)
}

email_diameter = diameter(email_g, directed = FALSE)
z_score = (email_diameter - mean(t1)) / sd(t1)
cat("Z-score:", z_score, "\n")

## Z-score: 21.31383
```

The high Z-score (>20) indicates that the diameter of the email network is statistically significantly larger than that of a comparable random network with the same degree distribution ($Z > 1.96$ is significant).